Solution: Spiral Matrix

Problem Statement

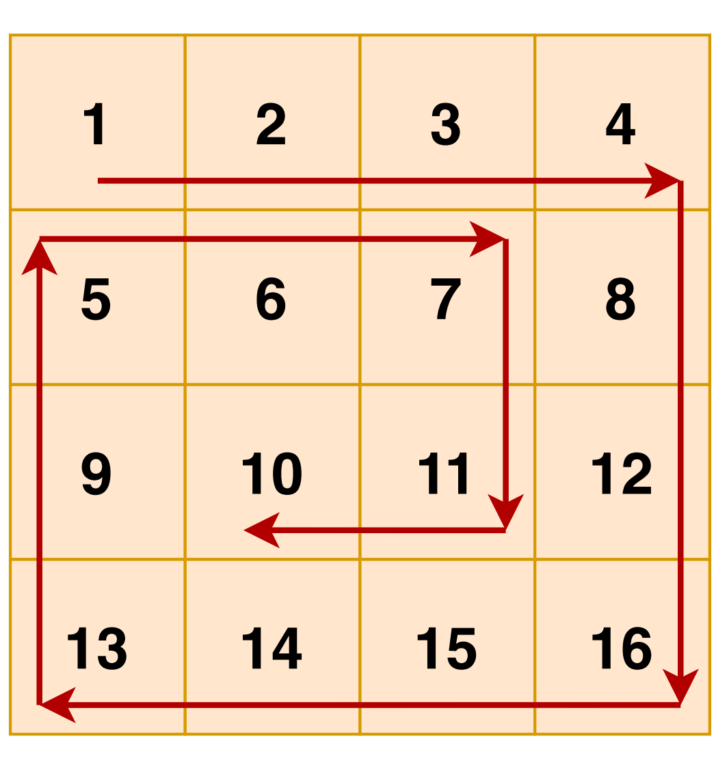
Given a 2D matrix of size `m x n`, return the 1D array containing all elements of matrix in `spiral order`.

Examples

Example 1:

- **Input**: matrix =

```
[[1,2,3,4],
 [5,6,7,8],
 [9,10,11,12],
 [13,14,15,16]]
```



- **Expected Output**: `[1,2,3,4,8,12,16,15,14,13,9,5,6,7,11,10]`
- **Justification**: We have traversed the matrix in the spiral order.

Example 2:

- **Input**: matrix =

```
[[10,20,30],
 [40,50,60],
 [70,80,90]]
```

- **Expected Output**: `[10,20,30,60,90,80,70,40,50]`
- **Justification**: The traversal starts at the top-left, moves right to the end of the row, then down the right column, then left along the bottom row, up the left column, and finally captures the center.

Example 3:

- **Input**: matrix =

```
[[1,2],
 [3,4],
 [5,6]]
```

- **Expected Output**: `[1,2,4,6,5,3]`
- **Justification**: The traversal starts at the top-left, moves right, then down through the right column, and finally moves up the left column.

Solution

To solve this problem, we employ a systematic approach to traverse the matrix in layers, starting from the outermost layer and gradually moving towards the inner layers. We maintain four pointers to mark the boundaries of the current layer: top, bottom, left, and right. Initially, these pointers are set to the boundaries of the matrix. In each iteration, we traverse the current layer in a spiral order: from left to right, top to bottom, right to left, and bottom to top, adjusting the boundary pointers accordingly after each direction of traversal. This approach ensures that we visit each element exactly once, maintaining the spiral order.

This method is effective because it clearly defines the traversal order and boundaries, preventing overlaps or missed elements. By systematically shrinking the traversal area and moving inward, we ensure complete coverage of the matrix in a spiral pattern. This logical progression mirrors the spiral traversal requirement, making it a natural fit for solving the problem.

Step-by-step Algorithm

- Initialize four pointers: `top = 0`, `bottom = matrix.length - 1`, `left = 0`, `right = matrix[0].length - 1`.
- Create an empty list `result` to store the spiral order.
- While `(top <= bottom && left <= right)`:
  - Traverse from `left` to `right` at the `top` row. After traversing, increment `top` pointer.
  - Traverse from `top` to `bottom` at the `right` column. After traversing, decrement `right` pointer.
  - If `(top <= bottom)`, traverse from `right` to `left` at the `bottom` row. After traversing, decrement `bottom` pointer.
  - If `(left <= right)`, traverse from `bottom` to `top` at the `left` column. After traversing, increment `left` pointer.
- Return the `result` list containing the elements in spiral order.

Algorithm Walkthrough

Let's consider the input:

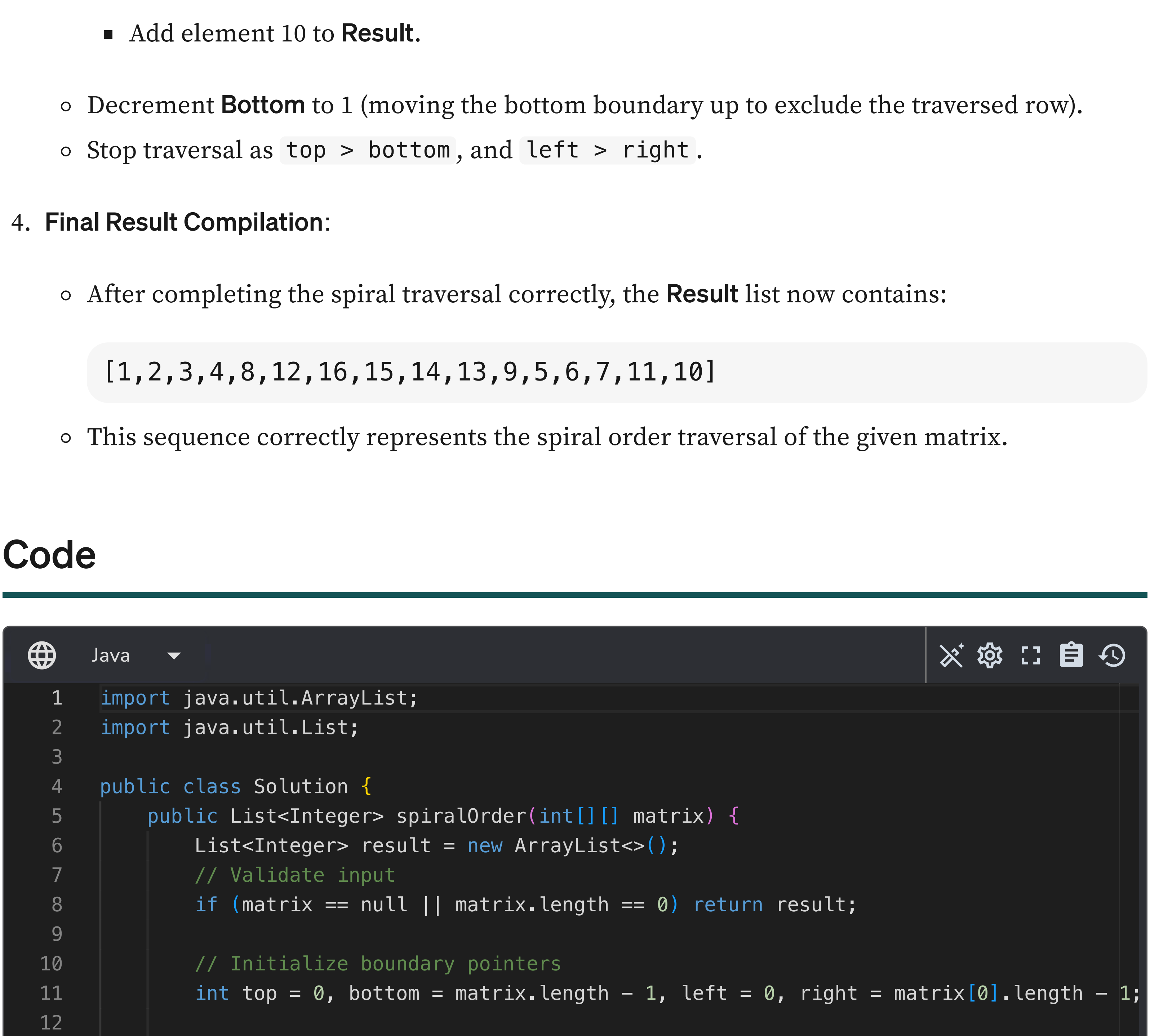
```
[[1,2,3,4],
 [5,6,7,8],
 [9,10,11,12],
 [13,14,15,16]]
```

1. **Initialize Boundary Pointers and Result Container**:
   - **Top** = 0 (index of the first row)
   - **Bottom** = 3 (index of the last row)
   - **Left** = 0 (index of the first column)
   - **Right** = 3 (index of the last column)
   - **Result** = [] (an empty list to store the spiral order)
2. **First Outer Layer Traversal**:
   - Traverse from **Left** to **Right** along the **Top** row:
     - Add elements 1, 2, 3, and 4 to **Result.**
   - Increment **Top** to 1 (moving the top boundary down to exclude the traversed row).
   - Traverse from **Top** to **Bottom** along the **Right** column:
     - Add elements 8, 12, and 16 to **Result.**
   - Decrement **Right** to 2 (moving the right boundary left to exclude the traversed column).
   - Since **Top** ≤ **Bottom**, traverse from **Right** to **Left** along the **Bottom** row:
     - Add elements 15, 14, and 13 to **Result.**
   - Decrement **Bottom** to 2 (moving the bottom boundary up to exclude the traversed row).
   - Since **Left** ≤ **Right**, traverse from **Bottom** to **Top** along the **Left** column:
     - Add elements 9 and 5 to **Result.**
   - Increment **Left** to 1 (moving the left boundary right to exclude the traversed column).
3. **Second Inner Layer Traversal**:
   - Now, the pointers are adjusted to: **Top** = 1, **Bottom** = 2, **Left** = 1, **Right** = 2.
   - Traverse from **Left** to **Right** along the new **Top** row (which is the second row now):
     - Add elements 6 and 7 to **Result.**
   - Increment **Top** to 2.
   - Traverse from **Top** to **Bottom** along the **Right** column:
     - Add element **11** in the results.
   - Decrement **Right** to 1 (moving the right boundary left to exclude the traversed column).
   - Since **Top** ≤ **Bottom**, traverse from **Right** to **Left** along the **Bottom** row:
     - Add element 10 to **Result.**
   - Decrement **Bottom** to 1 (moving the bottom boundary up to exclude the traversed row).
   - Stop traversal as `top > bottom`, and `left > right`.
4. **Final Result Compilation**:
   - After completing the spiral traversal correctly, the **Result** list now contains:

     ```
     [1,2,3,4,8,12,16,15,14,13,9,5,6,7,11,10]
     ```

   - This sequence correctly represents the spiral order traversal of the given matrix.

Code



```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> result = new ArrayList<>();
        // Validate input
        if (matrix == null || matrix.length == 0) return result;

        // Initialize boundary pointers
        int top = 0, bottom = matrix.length - 1, left = 0, right = matrix[0].length - 1;

        // Traverse the matrix in a spiral order
        while (top <= bottom && left <= right) {
            // Traverse from left to right
            for (int i = left; i <= right; i++) result.add(matrix[top][i]);
            top++;

            // Traverse from top to bottom
            for (int i = top; i <= bottom; i++) result.add(matrix[i][right]);
            right--;
```

You must run your code first

Complexity Analysis

Time Complexity

- $O(m*n)$, where `m * n` is the total number of elements in the matrix. Each element is visited exactly once, making the time complexity linear in terms of the number of elements in the matrix.

Space Complexity

- $O(1)$, disregarding the output array. The algorithm uses a constant amount of space for the pointers (top, bottom, left, right) and the result list's space is not considered part of the algorithm's space complexity as it is required for the output.
- If we consider the output space, then the space complexity is $O(m*n)$, where `m * n` is the total number of elements in the matrix.